The four data professions, end to end

Who owns which stage from raw data to a decision, and how each role is built

M Usman

An applied view of the data career, stage by stage.

Where this is heading

Four roles, one value chain, and the boundaries are blurring.

M Usman · From enterprise data to production AI.

The demand is clear; the titles are not

- AI and big-data roles are among the **fastest-growing to 2030** (WEF, 2025), and the measured global AI wage premium has reached **56%** (PwC, 2025).
- The confusion is in the labels, not the work:
  - The same four titles, **architect, engineer, analyst, scientist**, mean different things at different employers
  - A posting titled one role often describes another, so candidates mis-apply and teams mis-hire
  - Two skills now form the **floor in nearly every data role**: Python and SQL. Cloud has moved from a differentiator to an expectation, and generative AI, **LLMs and retrieval-augmented generation, is the fastest-rising requirement** across all four

> Stop reading the title. Read the stage of the work it owns.

The slow step

The titles drift; the stages do not

A unit of data travels the same road at every company: it is designed, it is moved, it is explained, and it is used to predict. Four professions own four stages of that road. The labels slide around, but the stages are stable, and each stage has one job and one common way to lose the value.

> Define a data role by the stage it owns and the decision it serves, not by the word in the job title. The four that follow are that road, in order.

The value chain, end to end

1 · Architect
design the foundation
leaks as: a blueprint nobody builds to

›

2 · Engineer
build and move the data
leaks as: silent breakage

›

3 · Analyst
explain what happened
leaks as: a chart with no decision

›

4 · Scientist
predict what to do
leaks as: leakage and vanity metrics

The numbering is the order the data moves, not a seniority or pay ladder. The usual career ladder does not follow it: analyst is the common entry point, scientist and engineer sit above, and architect is a senior destination reached after years in the chain. Seniority still crosses the lines, so an experienced analyst can out-earn a newly hired scientist, but that is the exception, not the rule. A real team needs all four, and the next four slides take each in turn.

1 · Data Architect: design the foundation

Owns the blueprint: how data is modelled, governed, and stored across the estate, so every downstream consumer reads data with a known shape, owner, and meaning.

- **Ships:** the target-state data model and reference architecture; master-data, security, and governance standards; the semantic and metrics layer defined once; the platform and tooling decision.
- **Measured by:** adoption of the standard (one model, not five); governance and data-quality coverage; time to onboard a new source; fewer duplicate or contradictory records; platform cost and performance.
- **Stack** (illustrative): dimensional and 3NF modelling; master-data management; metadata, catalogue, and lineage; warehouse and lakehouse patterns; data-mesh and domain ownership; the frameworks (TOGAF, DAMA-DMBOK).
- **Built from:** SQL and database internals, then data modelling and warehousing, then deep cloud-platform and governance practice. Usually reached from senior engineering or long enterprise-data experience, rarely entry level.

The pitfall: a blueprint nobody builds to. Governance designed on a slide and bolted on last is brittle and ignored; attached at source, it is enforced for free downstream.

2 · Data Engineer: build and move the data

Owns the pipelines: source data moved into layered, tested, business-ready tables under an orchestrator that manages dependencies, retries, and alerts.

- **Ships:** ingestion and transformation pipelines (raw, cleaned, modelled); data-quality tests as code; orchestrated dependency graphs with monitoring; the warehouse or lakehouse tables the rest of the org consumes.
- **Measured by:** pipeline reliability and freshness SLAs met; quality incidents caught before users see them; time to root-cause a break; cost per pipeline; latency of fresh data.
- **Stack** (illustrative): Python and SQL as the floor; Spark for scale; orchestration (Airflow, Dagster, Prefect); dbt for transformation; a warehouse or lakehouse (Snowflake, Databricks, BigQuery); streaming (Kafka, Iceberg); CI/CD and data observability.
- **Built from:** Python and SQL, then data-engineering fundamentals (pipelines, ETL and ELT, storage), then orchestration and dbt, then warehouse depth and streaming.

The pitfall: silent breakage. An upstream column is renamed or a load arrives half-empty; with no contract test the pipeline fails at 3am or, worse, propagates corrupted data downstream.

3 · Data Analyst: explain what happened

Owns the business question: governed data turned into a clear answer about what happened and why, placed in front of a decision.

- **Ships:** trusted metric definitions and dashboards; analyses that answer one specific question; experiment read-outs; the narrative that turns a number into an action.
- **Measured by:** decisions influenced and questions answered; trust and adoption of the metrics; time to answer; usage that leads to action rather than vanity views; the accuracy of the definition behind a number.
- **Stack** (illustrative): SQL as the floor; a BI tool (Power BI, Tableau, Looker); spreadsheet fluency; Python and pandas for deeper analysis; the shared semantic layer; descriptive statistics and the reading of an experiment.
- **Built from:** SQL, then statistics fundamentals, then BI and metric design, then Python and pandas, then experiment design and the analytics-engineering bridge (dbt) toward the engineer or scientist roles.

The pitfall: a chart with no decision. A dashboard nobody acts on, or a single metric three teams each define differently, produces activity and no value. An analysis is finished when it changes what someone does.

4 · Data Scientist: predict what to do

Owns the prediction: governed data turned into a model whose reported score is the score it will post in production, wired to a decision.

- **Ships:** trained, leakage-audited models with honest evaluation; experiments and their read-outs; engineered features; increasingly, models served behind a versioned interface (the MLOps overlap).
- **Measured by:** the decision metric, precision and recall or PR-AUC, not raw accuracy; whether the model clears a real baseline; production lift against the offline score; calibration; whether it ships and stays honest under drift.
- **Stack** (illustrative): Python and statistical rigour as the floor; scikit-learn, PyTorch, TensorFlow; the specialisms (time-series, graph ML and GNNs, NLP); LLMs, generative AI, and RAG as the rising edge; MLflow and evaluation discipline.
- **Built from:** Python, mathematics, and model evaluation, then ML fundamentals, then a specialism (time-series and forecasting, gradient boosting, NLP), then LLMs (engineering, LLM workflows and RAG, quantisation), then MLOps and vector search.

The pitfall: leakage and vanity metrics. Information from the future or the target slips into training; the score is spectacular and fictional, then collapses in production. Treat a result that looks too good as an alarm.

Where the boundaries blur

- **The floor is shared.** Python and SQL sit under all four. Cloud is now an expectation in every column, not an edge skill. Treat them as table stakes, not as a specialism.
- **The frontier is common.** LLMs and RAG are climbing fastest in every column, and the place the four roles are converging.
- **The hybrids are real.** The analytics engineer sits between analyst and engineer (dbt, the metrics layer); the ML engineer sits between scientist and engineer (serving, MLOps); the full-stack data scientist spans modelling to deployment. The title is a local convention.
- **The reading rule.** When the title is ambiguous, the stack and the deliverables on the posting tell you which stage it really owns. Match the work, not the word.

The shared roadmap every data role climbs

5 · Production and frontier
MLOps and deployment; LLMs, RAG, and vector search; monitoring and the served interface

4 · Modelling and honest evaluation
ML fundamentals; leakage-safe protocols; the metric chosen from the decision, not the leaderboard

3 · Pipelines and the cloud
ingest, transform, test; ETL and ELT; orchestration; a warehouse or lakehouse on one cloud

2 · Data and SQL
databases, SQL, and data modelling; the shape and meaning of the data

1 · Foundations
Python, version control, and the mathematics and statistics under everything above

All four roles start on the same first two rungs and branch near the top: the architect climbs through data and governance, the engineer through pipelines, the analyst through SQL and statistics, the scientist through modelling and the frontier.

What good looks like

- **Hire and learn by the stage, not the title.** Decide which stage you need owned, design, build, explain, or predict, and read the stack and deliverables to confirm the posting matches it.
- **Do not skip the floor.** Python, SQL, and one cloud are non-negotiable in every column; a strong specialism on a weak floor does not survive a technical screen.
- **Lead with the rising edge where it fits.** Cloud fluency and a real, evaluated LLM or RAG project are where a candidate stands out across all four roles right now, because that is where every column is moving.
- **Respect the handover.** Value is lost between the stages more often than inside them: the architect's contract, the engineer's tests, the analyst's definition, and the scientist's evaluation are the seams to protect.

The lens behind this view

My view comes from both ends of this chain, which are usually held apart. One end is eighteen years building and governing enterprise data estates across multiple ERP stacks. There, master data, the modelled layer, and the BI on top are the architect's, engineer's, and analyst's real problems rather than diagrams. The other end is doctoral work on efficient, proactive AI, where honest evaluation and the cost of a wrong prediction are the whole of the scientist's job. I also teach the database and data-mining foundations all four roles are built on.

So the first three stages are the daily work of those eighteen years, and the fourth is the work of the doctorate. The map above is where I have watched value get made, and lost.

> The research and the worked examples behind this view are public: mtusman-ai.github.io/M.Usman

In one line

A data role is the stage of the road it owns, not the word in its title. Design the foundation, build and move the data, explain what happened, predict what to do. Read the stack, not the title, and protect the seams between the four.

Sources

WEF Future of Jobs (2025): AI and big-data among the fastest-growing roles to 2030 · PwC Global AI Jobs Barometer (2025): 56% global AI wage premium · The analytics-maturity progression (descriptive → diagnostic → predictive → prescriptive) is the standard Gartner analytics-capability framing. The per-role learning paths follow the standard curriculum progression: foundations, then SQL and databases, then data engineering, then model evaluation and specialised ML, then LLM workflows and MLOps. Skill-demand signal drawn from a 2026 sweep of register-confirmed UK sponsor postings across the engineer, architect, and scientist families. Tools named are illustrative, not endorsements.

M Usman · From enterprise data to production AI. · mtusman-ai.github.io/M.Usman